```
################################################################################
#                                                                              #
#                 SQL QUESTION BANK - COMBINED ANSWERS (MySQL 8.0)             #
#                                                                              #
#   SET 1 : Questions 1-25  (Vehicle, Cinema, Courier, ... Scholarship)        #
#   SET 2 : Questions 1-25  (Student, Library, Employee, ... Online Shopping)  #
#                                                                              #
################################################################################

HOW TO USE
----------
1. Use MySQL 8.0 or a compatible version.
2. Run each question separately. Each question creates and uses its own database.
3. For authorization questions, run CREATE USER / GRANT / REVOKE as an
   administrator, then reconnect as the named user to test the statements.
4. Statements marked "should fail" are intentional demonstrations of
   constraint or permission violations.
5. CHECK constraints are enforced from MySQL 8.0.16 onward.
6. Transaction questions require the InnoDB storage engine.
7. If a user already exists, run  DROP USER IF EXISTS 'username'@'localhost';
   as administrator before CREATE USER.
8. DELIMITER works in the MySQL command-line client and MySQL Workbench.
   Some third-party SQL editors handle delimiters differently.

NOTE ON DATABASE NAMES
============================================================
SET 1 - Q1. VEHICLE RENTAL MANAGEMENT SYSTEM
============================================================
CREATE DATABASE vehicle_db;
USE vehicle_db;

CREATE TABLE Vehicle (
    VehicleID          INT PRIMARY KEY,
    VehicleName        VARCHAR(100) NOT NULL,
    RegistrationNo     VARCHAR(30) UNIQUE,
    RentPerDay         DECIMAL(10,2),
    AvailabilityStatus VARCHAR(20),
    CHECK (RentPerDay > 0)
);

ALTER TABLE Vehicle ADD VehicleType VARCHAR(50);
ALTER TABLE Vehicle RENAME COLUMN VehicleType TO Category;
DESCRIBE Vehicle;

-- Demonstrate UNIQUE constraint violation:
INSERT INTO Vehicle VALUES (1, 'Swift', 'MH01AB1234', 1500, 'Available', 'Car');

-- This second INSERT should fail because RegistrationNo must be unique.
INSERT INTO Vehicle VALUES (2, 'Baleno', 'MH01AB1234', 1700, 'Available', 'Car');


============================================================
SET 1 - Q2. CINEMA TICKET BOOKING SYSTEM
============================================================
CREATE DATABASE cinema_db;
USE cinema_db;

CREATE TABLE Movie (
    MovieID     INT PRIMARY KEY,
    MovieName   VARCHAR(50) NOT NULL,
    Language    VARCHAR(30),
    Duration    INT CHECK (Duration > 0),
    TicketPrice DECIMAL(8,2) DEFAULT 200
);

ALTER TABLE Movie ADD Genre VARCHAR(40);
ALTER TABLE Movie MODIFY MovieName VARCHAR(100);
DESCRIBE Movie;


============================================================
SET 1 - Q3. COURIER DELIVERY MANAGEMENT SYSTEM
```

```
============================================================
CREATE DATABASE courier_db;
USE courier_db;

CREATE TABLE Sender (
    SenderID   INT PRIMARY KEY,
    SenderName VARCHAR(100) NOT NULL,
    MobileNo   VARCHAR(15)
);

CREATE TABLE Parcel (
    ParcelID       INT PRIMARY KEY,
    SenderID       INT,
    Weight         DECIMAL(8,2) CHECK (Weight > 0),
    DeliveryStatus VARCHAR(30) DEFAULT 'Pending',
    FOREIGN KEY (SenderID) REFERENCES Sender(SenderID)
);

ALTER TABLE Parcel ADD DestinationCity VARCHAR(60);

INSERT INTO Sender VALUES (1, 'Amit Sharma', '9876543210');

-- This INSERT should fail because SenderID 99 does not exist in Sender.
INSERT INTO Parcel (ParcelID, SenderID, Weight, DestinationCity)
VALUES (101, 99, 2.5, 'Pune');


============================================================
SET 1 - Q4. SPORTS ACADEMY MANAGEMENT SYSTEM
============================================================
CREATE DATABASE sports_db;
USE sports_db;

CREATE TABLE Coach (
    CoachID        INT PRIMARY KEY,
    CoachName      VARCHAR(100),
    Specialization VARCHAR(60)
);

CREATE TABLE Player (
    PlayerID   INT PRIMARY KEY,
    PlayerName VARCHAR(100),
    Age        INT CHECK (Age >= 10),
    CoachID    INT,
    FOREIGN KEY (CoachID) REFERENCES Coach(CoachID)
);

ALTER TABLE Player ADD JoiningDate DATE;
ALTER TABLE Coach RENAME COLUMN Specialization TO SportType;

DESCRIBE Coach;
DESCRIBE Player;


============================================================
SET 1 - Q5. PHARMACY INVENTORY MANAGEMENT
============================================================
CREATE DATABASE pharmacy_db;
USE pharmacy_db;

CREATE TABLE Medicine (
    MedicineID   INT PRIMARY KEY,
    MedicineName VARCHAR(100),
    Price        DECIMAL(10,2),
    Quantity     INT,
    ExpiryDate   DATE
);

INSERT INTO Medicine VALUES
(1, 'Paracetamol',  25.00, 100, '2027-05-31'),
(2, 'Cetirizine',   40.00,  80, '2027-08-15'),
(3, 'Amoxicillin',  90.00,  50, '2027-01-10'),
```

```
(4, 'Vitamin C',    60.00,  70, '2028-03-20'),
(5, 'Cough Syrup', 110.00,  40, '2026-09-01');

UPDATE Medicine SET Price = Price * 1.08 WHERE MedicineID = 1;
UPDATE Medicine SET Quantity = Quantity - 10 WHERE MedicineID = 2;

DELETE FROM Medicine WHERE ExpiryDate < CURDATE();

SELECT * FROM Medicine;


============================================================
SET 1 - Q6. RESTAURANT MENU MANAGEMENT
============================================================
CREATE DATABASE restaurant_db;
USE restaurant_db;

CREATE TABLE Menu (
    ItemID       INT PRIMARY KEY,
    ItemName     VARCHAR(100),
    Category     VARCHAR(50),
    Price        DECIMAL(8,2),
    Availability VARCHAR(20)
);

INSERT INTO Menu VALUES
(1, 'Tea',          'Beverages',   20,  'Available'),
(2, 'Coffee',       'Beverages',   50,  'Available'),
(3, 'Lemon Soda',   'Beverages',   60,  'Available'),
(4, 'Veg Sandwich', 'Snacks',      80,  'Available'),
(5, 'Paneer Pizza', 'Main Course', 220, 'Unavailable'),
(6, 'French Fries', 'Snacks',      100, 'Available');

UPDATE Menu SET Price = Price * 1.05 WHERE Category = 'Beverages';

DELETE FROM Menu WHERE ItemID = 5 AND Availability = 'Unavailable';

SELECT * FROM Menu WHERE Availability = 'Available';


============================================================
SET 1 - Q7. GYM MEMBERSHIP MANAGEMENT
============================================================
CREATE DATABASE gym_db;
USE gym_db;

CREATE TABLE Member (
    MemberID       INT PRIMARY KEY,
    MemberName     VARCHAR(100),
    MembershipType VARCHAR(30),
    Fees           DECIMAL(10,2),
    Status         VARCHAR(20)
);

INSERT INTO Member VALUES
(1, 'Rahul', 'Basic',    2000, 'Active'),
(2, 'Priya', 'Standard', 3000, 'Active'),
(3, 'Aman',  'Basic',    2000, 'Inactive'),
(4, 'Neha',  'Standard', 3000, 'Active'),
(5, 'Karan', 'Basic',    2000, 'Active');

UPDATE Member
SET MembershipType = 'Premium', Fees = Fees + 1000
WHERE MemberID = 2;

DELETE FROM Member WHERE Status = 'Inactive';

SELECT * FROM Member WHERE Status = 'Active';


============================================================
SET 1 - Q8. MOBILE RECHARGE MANAGEMENT SYSTEM
```

```
============================================================
CREATE DATABASE recharge_db;
USE recharge_db;

CREATE TABLE Recharge (
    RechargeID     INT PRIMARY KEY,
    MobileNo       VARCHAR(15),
    Operator       VARCHAR(30),
    Amount         DECIMAL(8,2),
    RechargeStatus VARCHAR(20)
);

INSERT INTO Recharge VALUES
(1, '9876500001', 'Jio',    299, 'Successful'),
(2, '9876500002', 'Airtel', 349, 'Pending'),
(3, '9876500003', 'Vi',     199, 'Failed'),
(4, '9876500004', 'BSNL',   187, 'Successful'),
(5, '9876500005', 'Jio',    399, 'Pending');

UPDATE Recharge SET Amount = 449 WHERE RechargeID = 5;

UPDATE Recharge
SET RechargeStatus = 'Successful'
WHERE RechargeID = 2 AND RechargeStatus = 'Pending';

DELETE FROM Recharge WHERE RechargeID = 3 AND RechargeStatus = 'Failed';

SELECT * FROM Recharge WHERE RechargeStatus = 'Successful';


============================================================
SET 1 - Q9. SCHOOL DATABASE AUTHORIZATION
============================================================
CREATE DATABASE school_db;
USE school_db;

CREATE TABLE Teacher (
    TeacherID   INT PRIMARY KEY,
    TeacherName VARCHAR(100),
    Subject     VARCHAR(50)
);

INSERT INTO Teacher VALUES
(1, 'Anita',  'Mathematics'),
(2, 'Ramesh', 'Science');

-- Run as administrator:
CREATE USER 'school_user'@'localhost' IDENTIFIED BY 'School@123';
GRANT SELECT ON school_db.Teacher TO 'school_user'@'localhost';

-- Connect separately as school_user:
SELECT * FROM school_db.Teacher;                      -- Allowed
DELETE FROM school_db.Teacher WHERE TeacherID = 1;    -- Denied (should fail)

-- Reconnect as administrator:
REVOKE SELECT ON school_db.Teacher FROM 'school_user'@'localhost';
SHOW GRANTS FOR 'school_user'@'localhost';


============================================================
SET 1 - Q10. RAILWAY RESERVATION AUTHORIZATION
============================================================
CREATE DATABASE railway_db;
USE railway_db;

CREATE TABLE Train (
    TrainID        INT PRIMARY KEY,
    TrainName      VARCHAR(100),
    AvailableSeats INT
);

INSERT INTO Train VALUES
```

```
(101, 'Deccan Express',    100),
(102, 'Intercity Express',  80);

-- Run as administrator:
CREATE USER 'booking_user'@'localhost' IDENTIFIED BY 'Booking@123';
GRANT SELECT, UPDATE ON railway_db.Train TO 'booking_user'@'localhost';

-- Connect separately as booking_user:
SELECT * FROM railway_db.Train;
UPDATE railway_db.Train SET AvailableSeats = 98 WHERE TrainID = 101;   -- Allowed
INSERT INTO railway_db.Train VALUES (103, 'New Express', 60);          -- Denied (should fail)

-- Reconnect as administrator:
REVOKE UPDATE ON railway_db.Train FROM 'booking_user'@'localhost';
SHOW GRANTS FOR 'booking_user'@'localhost';

-- Connect as booking_user again; this should fail:
UPDATE railway_db.Train SET AvailableSeats = 97 WHERE TrainID = 101;   -- Denied


============================================================
SET 1 - Q11. RETAIL STORE AUTHORIZATION
============================================================
CREATE DATABASE retail_db;
USE retail_db;

CREATE TABLE Inventory (
    ProductID   INT PRIMARY KEY,
    ProductName VARCHAR(100),
    Quantity    INT
);

INSERT INTO Inventory VALUES (1, 'Notebook', 100);

-- Run as administrator:
CREATE USER 'store_user'@'localhost' IDENTIFIED BY 'Store@123';
GRANT SELECT, INSERT ON retail_db.Inventory TO 'store_user'@'localhost';

-- Connect separately as store_user:
INSERT INTO retail_db.Inventory VALUES (2, 'Pen', 200);                -- Allowed
UPDATE retail_db.Inventory SET Quantity = 150 WHERE ProductID = 1;     -- Denied (should fail)

-- Reconnect as administrator:
REVOKE INSERT ON retail_db.Inventory FROM 'store_user'@'localhost';
SHOW GRANTS FOR 'store_user'@'localhost';


============================================================
SET 1 - Q12. EXAMINATION DATABASE AUTHORIZATION
============================================================
CREATE DATABASE exam_db;
USE exam_db;

CREATE TABLE Result (
    ResultID    INT PRIMARY KEY,
    StudentName VARCHAR(100),
    Marks       INT
);

INSERT INTO Result VALUES
(1, 'Asha',  85),
(2, 'Vijay', 72);

-- Run as administrator:
CREATE USER 'exam_user'@'localhost' IDENTIFIED BY 'Exam@123';
GRANT SELECT, UPDATE ON exam_db.Result TO 'exam_user'@'localhost';

-- Connect separately as exam_user:
UPDATE exam_db.Result SET Marks = 90 WHERE ResultID = 1;               -- Allowed
DELETE FROM exam_db.Result WHERE ResultID = 2;                         -- Denied (should fail)

-- Reconnect as administrator:
```

```
REVOKE UPDATE ON exam_db.Result FROM 'exam_user'@'localhost';
SHOW GRANTS FOR 'exam_user'@'localhost';


============================================================
SET 1 - Q13. AIRLINE TICKET ANALYSIS
============================================================
CREATE DATABASE airline_db;
USE airline_db;

CREATE TABLE Flight (
    FlightID       INT PRIMARY KEY,
    AirlineName    VARCHAR(100),
    Destination    VARCHAR(60),
    TicketPrice    DECIMAL(10,2),
    AvailableSeats INT
);

INSERT INTO Flight VALUES
(1, 'Air India', 'Delhi',     4500, 30),
(2, 'IndiGo',    'Mumbai',    3200, 25),
(3, 'Vistara',   'Bengaluru', 6200, 20),
(4, 'Air India', 'Mumbai',    4800, 18),
(5, 'IndiGo',    'Delhi',     3900, 22),
(6, 'SpiceJet',  'Pune',      2800, 15),
(7, 'Vistara',   'Bengaluru', 5500, 12),
(8, 'Akasa Air', 'Pune',      3500, 16);

-- Tickets below 5000, cheapest first.
SELECT * FROM Flight
WHERE TicketPrice < 5000
ORDER BY TicketPrice ASC;

-- Maximum, minimum and average price.
SELECT MAX(TicketPrice) AS MaximumPrice,
       MIN(TicketPrice) AS MinimumPrice,
       AVG(TicketPrice) AS AveragePrice
FROM Flight;

-- Average price by destination.
SELECT Destination, AVG(TicketPrice) AS AverageTicketPrice
FROM Flight
GROUP BY Destination;


============================================================
SET 1 - Q14. HOSPITAL DOCTOR ANALYSIS
============================================================
CREATE DATABASE hospital_db;
USE hospital_db;

CREATE TABLE Doctor (
    DoctorID        INT PRIMARY KEY,
    DoctorName      VARCHAR(100),
    Specialization  VARCHAR(60),
    ConsultationFee DECIMAL(10,2)
);

INSERT INTO Doctor VALUES
(1, 'Dr. Mehta', 'Cardiology',  1200),
(2, 'Dr. Shah',  'Cardiology',  1000),
(3, 'Dr. Rao',   'Neurology',   1500),
(4, 'Dr. Iyer',  'Neurology',    900),
(5, 'Dr. Khan',  'Pediatrics',   700),
(6, 'Dr. Das',   'Pediatrics',   600),
(7, 'Dr. Patil', 'Orthopedics', 1100),
(8, 'Dr. Roy',   'Orthopedics',  850);

-- Doctors with fee above 800.
SELECT * FROM Doctor WHERE ConsultationFee > 800;

-- Number of doctors and average fee per specialization.
```

```
SELECT Specialization,
       COUNT(*) AS NumberOfDoctors,
       AVG(ConsultationFee) AS AverageFee
FROM Doctor
GROUP BY Specialization;

-- Doctors charging more than the overall average.
SELECT * FROM Doctor
WHERE ConsultationFee > (SELECT AVG(ConsultationFee) FROM Doctor);


============================================================
SET 1 - Q15. E-COMMERCE SALES ANALYSIS
============================================================
CREATE DATABASE ecommerce_db;
USE ecommerce_db;

CREATE TABLE Category (
    CategoryID   INT PRIMARY KEY,
    CategoryName VARCHAR(60)
);

CREATE TABLE Product (
    ProductID   INT PRIMARY KEY,
    ProductName VARCHAR(100),
    Price       DECIMAL(10,2),
    CategoryID  INT,
    FOREIGN KEY (CategoryID) REFERENCES Category(CategoryID)
);

INSERT INTO Category VALUES
(1, 'Electronics'),
(2, 'Clothing'),
(3, 'Books'),
(4, 'Home Appliances');

INSERT INTO Product VALUES
(1, 'Smartphone', 15000, 1),
(2, 'Headphones',  1800, 1),
(3, 'T-Shirt',      700, 2),
(4, 'Jeans',       1400, 2),
(5, 'Novel',        500, 3),
(6, 'Textbook',    1200, 3),
(7, 'Mixer',       2500, 4),
(8, 'Microwave',   8500, 4);

-- Products with category names.
SELECT p.ProductName, p.Price, c.CategoryName
FROM Product p
INNER JOIN Category c ON p.CategoryID = c.CategoryID;

-- Average price per category.
SELECT c.CategoryName, AVG(p.Price) AS AveragePrice
FROM Category c
INNER JOIN Product p ON c.CategoryID = p.CategoryID
GROUP BY c.CategoryID, c.CategoryName;

-- Categories with average price above 1000.
SELECT c.CategoryName, AVG(p.Price) AS AveragePrice
FROM Category c
INNER JOIN Product p ON c.CategoryID = p.CategoryID
GROUP BY c.CategoryID, c.CategoryName
HAVING AVG(p.Price) > 1000;


============================================================
SET 1 - Q16. COURSE ENROLLMENT ANALYSIS
============================================================
CREATE DATABASE course_db;
USE course_db;

CREATE TABLE Course (
```

```
    CourseID   INT PRIMARY KEY,
    CourseName VARCHAR(100),
    Fees       DECIMAL(10,2)
);

CREATE TABLE Enrollment (
    EnrollmentID INT PRIMARY KEY,
    StudentName  VARCHAR(100),
    CourseID     INT,
    FOREIGN KEY (CourseID) REFERENCES Course(CourseID)
);

INSERT INTO Course VALUES
(1, 'DBMS',            5000),
(2, 'Java',            6000),
(3, 'Python',          5500),
(4, 'Web Development', 7000);

INSERT INTO Enrollment VALUES
(1, 'Aarav',  1),
(2, 'Diya',   1),
(3, 'Kabir',  1),
(4, 'Meera',  2),
(5, 'Rohan',  2),
(6, 'Ishita', 3),
(7, 'Arjun',  3),
(8, 'Sara',   4);

-- Students with their course names.
SELECT e.StudentName, c.CourseName
FROM Enrollment e
JOIN Course c ON e.CourseID = c.CourseID;

-- Number of students per course (includes courses with zero students).
SELECT c.CourseName, COUNT(e.EnrollmentID) AS StudentCount
FROM Course c
LEFT JOIN Enrollment e ON c.CourseID = e.CourseID
GROUP BY c.CourseID, c.CourseName;

-- Courses with more than 2 students.
SELECT c.CourseName, COUNT(e.EnrollmentID) AS StudentCount
FROM Course c
JOIN Enrollment e ON c.CourseID = e.CourseID
GROUP BY c.CourseID, c.CourseName
HAVING COUNT(e.EnrollmentID) > 2;


============================================================
SET 1 - Q17. REAL ESTATE PROPERTY ANALYSIS
============================================================
CREATE DATABASE realestate_db;
USE realestate_db;

CREATE TABLE Property (
    PropertyID   INT PRIMARY KEY,
    Location     VARCHAR(60),
    PropertyType VARCHAR(50),
    Price        DECIMAL(14,2),
    Area         INT
);

INSERT INTO Property VALUES
(1, 'Mumbai', 'Flat', 4500000,  700),
(2, 'Pune',   'Flat', 3500000,  850),
(3, 'Thane',  'Shop', 6000000,  500),
(4, 'Mumbai', 'Flat', 8000000,  950),
(5, 'Nashik', 'Plot', 2500000, 1200),
(6, 'Pune',   'Villa',7000000, 1500),
(7, 'Thane',  'Flat', 5500000,  800),
(8, 'Nashik', 'Flat', 4000000,  900);

-- Properties priced between 30 lakh and 70 lakh.
```

```
SELECT * FROM Property
WHERE Price BETWEEN 3000000 AND 7000000;

-- Property count and average price per location.
SELECT Location,
       COUNT(*) AS PropertyCount,
       AVG(Price) AS AveragePrice
FROM Property
GROUP BY Location;

-- Properties priced above the overall average.
SELECT * FROM Property
WHERE Price > (SELECT AVG(Price) FROM Property);


============================================================
SET 1 - Q18. INSURANCE POLICY ANALYSIS
============================================================
CREATE DATABASE insurance_db;
USE insurance_db;

CREATE TABLE Customer (
    CustomerID   INT PRIMARY KEY,
    CustomerName VARCHAR(100)
);

CREATE TABLE Policy (
    PolicyID   INT PRIMARY KEY,
    CustomerID INT,
    PolicyType VARCHAR(60),
    Premium    DECIMAL(10,2),
    FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID)
);

INSERT INTO Customer VALUES
(1, 'Amit'),
(2, 'Sneha'),
(3, 'Raj'),
(4, 'Pooja'),
(5, 'Kunal');

INSERT INTO Policy VALUES
(101, 1, 'Life',    12000),
(102, 1, 'Health',   8000),
(103, 2, 'Vehicle',  6000),
(104, 3, 'Life',    15000),
(105, 3, 'Health',   9000),
(106, 4, 'Travel',   3000);

-- Customers with their policies.
SELECT c.CustomerName, p.PolicyType, p.Premium
FROM Customer c
INNER JOIN Policy p ON c.CustomerID = p.CustomerID;

-- Total premium per customer (zero if no policy).
SELECT c.CustomerID, c.CustomerName,
       COALESCE(SUM(p.Premium), 0) AS TotalPremium
FROM Customer c
LEFT JOIN Policy p ON c.CustomerID = p.CustomerID
GROUP BY c.CustomerID, c.CustomerName;

-- Customers with no policy.
SELECT c.CustomerID, c.CustomerName
FROM Customer c
LEFT JOIN Policy p ON c.CustomerID = p.CustomerID
WHERE p.PolicyID IS NULL;


============================================================
SET 1 - Q19. HOTEL ROOM PRICE MONITORING
============================================================
CREATE DATABASE hotel_db;
```

```
USE hotel_db;

CREATE TABLE Room (
    RoomID             INT PRIMARY KEY,
    RoomType           VARCHAR(50),
    PricePerNight      DECIMAL(10,2),
    AvailabilityStatus VARCHAR(20)
);

INSERT INTO Room VALUES
(1, 'Single', 1500, 'Available'),
(2, 'Double', 2500, 'Available'),
(3, 'Deluxe', 4000, 'Booked');

CREATE VIEW available_rooms AS
SELECT * FROM Room WHERE AvailabilityStatus = 'Available';

CREATE TABLE RoomPriceAudit (
    AuditID   INT AUTO_INCREMENT PRIMARY KEY,
    RoomID    INT,
    OldPrice  DECIMAL(10,2),
    NewPrice  DECIMAL(10,2),
    ChangedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

DELIMITER //

CREATE TRIGGER room_price_audit
AFTER UPDATE ON Room
FOR EACH ROW
BEGIN
    IF OLD.PricePerNight <> NEW.PricePerNight THEN
        INSERT INTO RoomPriceAudit (RoomID, OldPrice, NewPrice)
        VALUES (OLD.RoomID, OLD.PricePerNight, NEW.PricePerNight);
    END IF;
END //

DELIMITER ;

UPDATE Room SET PricePerNight = 2800 WHERE RoomID = 2;

SELECT * FROM available_rooms;
SELECT * FROM RoomPriceAudit;


============================================================
SET 1 - Q20. STUDENT MARKS MONITORING
============================================================
CREATE DATABASE student_db;
USE student_db;

CREATE TABLE Student (
    StudentID   INT PRIMARY KEY,
    StudentName VARCHAR(100),
    Marks       INT,
    Department  VARCHAR(60)
);

INSERT INTO Student VALUES
(1, 'Asha', 82, 'Computer'),
(2, 'Ravi', 68, 'Mechanical'),
(3, 'Neha', 91, 'Computer');

CREATE VIEW distinction_students AS
SELECT * FROM Student WHERE Marks >= 75;

CREATE TABLE MarksAudit (
    AuditID   INT AUTO_INCREMENT PRIMARY KEY,
    StudentID INT,
    OldMarks  INT,
    NewMarks  INT,
    ChangedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

```
);

DELIMITER //

CREATE TRIGGER marks_audit
AFTER UPDATE ON Student
FOR EACH ROW
BEGIN
    IF OLD.Marks <> NEW.Marks THEN
        INSERT INTO MarksAudit (StudentID, OldMarks, NewMarks)
        VALUES (OLD.StudentID, OLD.Marks, NEW.Marks);
    END IF;
END //

DELIMITER ;

UPDATE Student SET Marks = 78 WHERE StudentID = 2;

SELECT * FROM distinction_students;
SELECT * FROM MarksAudit;


============================================================
SET 1 - Q21. ONLINE FOOD DELIVERY MONITORING
============================================================
CREATE DATABASE food_delivery_db;
USE food_delivery_db;

CREATE TABLE FoodOrder (
    OrderID      INT PRIMARY KEY,
    CustomerName VARCHAR(100),
    OrderAmount  DECIMAL(10,2),
    OrderStatus  VARCHAR(30)
);

INSERT INTO FoodOrder VALUES
(1, 'Amit',  450, 'Pending'),
(2, 'Priya', 600, 'Delivered'),
(3, 'Rahul', 320, 'Pending');

CREATE VIEW pending_orders AS
SELECT * FROM FoodOrder WHERE OrderStatus = 'Pending';

CREATE TABLE OrderStatusAudit (
    AuditID   INT AUTO_INCREMENT PRIMARY KEY,
    OrderID   INT,
    OldStatus VARCHAR(30),
    NewStatus VARCHAR(30),
    ChangedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

DELIMITER //

CREATE TRIGGER order_status_audit
AFTER UPDATE ON FoodOrder
FOR EACH ROW
BEGIN
    IF OLD.OrderStatus <> NEW.OrderStatus THEN
        INSERT INTO OrderStatusAudit (OrderID, OldStatus, NewStatus)
        VALUES (OLD.OrderID, OLD.OrderStatus, NEW.OrderStatus);
    END IF;
END //

DELIMITER ;

UPDATE FoodOrder
SET OrderStatus = 'Delivered'
WHERE OrderID = 1 AND OrderStatus = 'Pending';

SELECT * FROM pending_orders;
SELECT * FROM OrderStatusAudit;
```

```
============================================================
SET 1 - Q22. ELECTRICITY BILL MONITORING
============================================================
CREATE DATABASE electricity_db;
USE electricity_db;

CREATE TABLE ElectricityBill (
    BillID        INT PRIMARY KEY,
    ConsumerName  VARCHAR(100),
    UnitsConsumed INT,
    BillAmount    DECIMAL(10,2),
    PaymentStatus VARCHAR(20)
);

INSERT INTO ElectricityBill VALUES
(1, 'Amit',  120,  950, 'Unpaid'),
(2, 'Sneha', 200, 1500, 'Paid'),
(3, 'Raj',   150, 1100, 'Unpaid');

CREATE VIEW unpaid_bills AS
SELECT * FROM ElectricityBill WHERE PaymentStatus = 'Unpaid';

CREATE TABLE BillAudit (
    AuditID   INT AUTO_INCREMENT PRIMARY KEY,
    BillID    INT,
    OldAmount DECIMAL(10,2),
    NewAmount DECIMAL(10,2),
    ChangedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

DELIMITER //

CREATE TRIGGER bill_amount_audit
AFTER UPDATE ON ElectricityBill
FOR EACH ROW
BEGIN
    IF OLD.BillAmount <> NEW.BillAmount THEN
        INSERT INTO BillAudit (BillID, OldAmount, NewAmount)
        VALUES (OLD.BillID, OLD.BillAmount, NEW.BillAmount);
    END IF;
END //

DELIMITER ;

UPDATE ElectricityBill SET BillAmount = 1000 WHERE BillID = 1;

SELECT * FROM unpaid_bills;
SELECT * FROM BillAudit;


============================================================
SET 1 - Q23. RAILWAY SEAT RESERVATION TRANSACTION
============================================================
CREATE DATABASE railway_transaction_db;
USE railway_transaction_db;

CREATE TABLE Train (
    TrainID        INT PRIMARY KEY,
    TrainName      VARCHAR(100),
    AvailableSeats INT
) ENGINE = InnoDB;

INSERT INTO Train VALUES
(101, 'Deccan Express',     100),
(102, 'Intercity Express',   80),
(103, 'Rajdhani Express',    60);

-- Transaction 1: book 2 seats and COMMIT.
START TRANSACTION;
UPDATE Train SET AvailableSeats = AvailableSeats - 2 WHERE TrainID = 101;
COMMIT;
```

```
SELECT * FROM Train WHERE TrainID = 101;

-- Transaction 2: book 3 seats, then ROLLBACK.
START TRANSACTION;
UPDATE Train SET AvailableSeats = AvailableSeats - 3 WHERE TrainID = 102;
ROLLBACK;

-- Seat count for Train 102 remains unchanged because of ROLLBACK.
SELECT * FROM Train WHERE TrainID = 102;


============================================================
SET 1 - Q24. WAREHOUSE STOCK TRANSACTION
============================================================
CREATE DATABASE warehouse_db;
USE warehouse_db;

CREATE TABLE Warehouse (
    ProductID     INT PRIMARY KEY,
    ProductName   VARCHAR(100),
    StockQuantity INT
) ENGINE = InnoDB;

INSERT INTO Warehouse VALUES
(201, 'Laptop',   50),
(202, 'Keyboard', 100),
(203, 'Mouse',    150);

START TRANSACTION;

UPDATE Warehouse SET StockQuantity = StockQuantity + 20 WHERE ProductID = 201;

SAVEPOINT stock_savepoint;

UPDATE Warehouse SET StockQuantity = StockQuantity - 10 WHERE ProductID = 202;

ROLLBACK TO SAVEPOINT stock_savepoint;
COMMIT;

-- Product 201's increase is saved; Product 202's decrease is undone.
SELECT * FROM Warehouse;


============================================================
SET 1 - Q25. SCHOLARSHIP DISBURSEMENT TRANSACTION
============================================================
CREATE DATABASE scholarship_db;
USE scholarship_db;

CREATE TABLE Scholarship (
    StudentID         INT PRIMARY KEY,
    StudentName       VARCHAR(100),
    ScholarshipAmount DECIMAL(10,2),
    Status            VARCHAR(30)
) ENGINE = InnoDB;

INSERT INTO Scholarship VALUES
(101, 'Aarav', 10000, 'Approved'),
(102, 'Diya',  12000, 'Approved'),
(103, 'Kabir',  8000, 'Pending');

START TRANSACTION;

UPDATE Scholarship
SET ScholarshipAmount = ScholarshipAmount + 3000
WHERE StudentID = 101;

SAVEPOINT scholarship_savepoint;

UPDATE Scholarship
SET ScholarshipAmount = ScholarshipAmount + 2000
```

```
WHERE StudentID = 102;

ROLLBACK TO SAVEPOINT scholarship_savepoint;
COMMIT;

-- Only Student 101's increase is saved; Student 102's increase is undone.
SELECT * FROM Scholarship;

============================================================
SET 2 - Q1. STUDENT MANAGEMENT SYSTEM
============================================================
CREATE DATABASE student_mgmt_db;
USE student_mgmt_db;

CREATE TABLE Student (
    StudentID  INT PRIMARY KEY,
    Name       VARCHAR(50) NOT NULL,
    Email      VARCHAR(100) UNIQUE,
    Age        INT CHECK (Age >= 18),
    Department VARCHAR(50)
);

ALTER TABLE Student ADD MobileNo VARCHAR(15);
DESC Student;

INSERT INTO Student VALUES
(1, 'Amit', 'amit@gmail.com', 20, 'CSE', '9876543210');

-- Demonstrate a UNIQUE constraint violation (this statement should fail).
INSERT INTO Student VALUES
(2, 'Rahul', 'amit@gmail.com', 19, 'IT', '9876543211');

SELECT * FROM Student;


============================================================
SET 2 - Q2. LIBRARY MANAGEMENT SYSTEM
============================================================
CREATE DATABASE library_db;
USE library_db;

CREATE TABLE Book (
    BookID   INT PRIMARY KEY,
    Title    VARCHAR(100) NOT NULL,
    Author   VARCHAR(50),
    Price    DECIMAL(10,2) CHECK (Price > 0),
    Quantity INT DEFAULT 1
);

ALTER TABLE Book ADD Category VARCHAR(50);
ALTER TABLE Book RENAME COLUMN Category TO Genre;

DESC Book;

INSERT INTO Book (BookID, Title, Author, Price, Genre)
VALUES (1, 'DBMS', 'Korth', 500, 'Education');

SELECT * FROM Book;


============================================================
SET 2 - Q3. EMPLOYEE MANAGEMENT SYSTEM
============================================================
CREATE DATABASE employee_db;
USE employee_db;

CREATE TABLE Department (
    DeptID   INT PRIMARY KEY,
    DeptName VARCHAR(50)
);

CREATE TABLE Employee (
```

```
    EmpID   INT PRIMARY KEY,
    EmpName VARCHAR(50) NOT NULL,
    Salary  DECIMAL(10,2) CHECK (Salary > 0),
    DeptID  INT,
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
);

ALTER TABLE Employee ADD JoiningDate DATE;

INSERT INTO Department VALUES
(1, 'IT'),
(2, 'HR');

INSERT INTO Employee VALUES
(101, 'Amit', 50000, 1, '2026-01-10');

DESC Department;
DESC Employee;


============================================================
SET 2 - Q4. HOSPITAL MANAGEMENT SYSTEM
============================================================
CREATE DATABASE hospital_mgmt_db;
USE hospital_mgmt_db;

CREATE TABLE Patient (
    PatientID   INT PRIMARY KEY,
    PatientName VARCHAR(50) NOT NULL,
    Age         INT CHECK (Age > 0),
    Gender      VARCHAR(10)
);

CREATE TABLE Appointment (
    AppointmentID   INT PRIMARY KEY,
    PatientID       INT,
    AppointmentDate DATE,
    FOREIGN KEY (PatientID) REFERENCES Patient(PatientID)
);

ALTER TABLE Appointment ADD DoctorName VARCHAR(50);

INSERT INTO Patient VALUES
(1, 'Rahul', 25, 'Male');

INSERT INTO Appointment VALUES
(101, 1, '2026-11-05', 'Dr. Sharma');

DESC Patient;
DESC Appointment;


============================================================
SET 2 - Q5. ONLINE SHOPPING SYSTEM
============================================================
CREATE DATABASE shopping_db;
USE shopping_db;

CREATE TABLE Product (
    ProductID   INT PRIMARY KEY,
    ProductName VARCHAR(50),
    Price       DECIMAL(10,2),
    Stock       INT
);

INSERT INTO Product VALUES
(1, 'Laptop',     50000, 10),
(2, 'Mouse',        500, 20),
(3, 'Keyboard',    1000, 15),
(4, 'Monitor',    12000,  8),
(5, 'Headphones',  2000, 12);
```

```
-- Increase Laptop price by 10%.
UPDATE Product SET Price = Price * 1.10 WHERE ProductID = 1;

-- Reduce Mouse stock by 2.
UPDATE Product SET Stock = Stock - 2 WHERE ProductID = 2;

-- Delete Headphones.
DELETE FROM Product WHERE ProductID = 5;

SELECT * FROM Product;


============================================================
SET 2 - Q6. BANK ACCOUNT MANAGEMENT
============================================================
CREATE DATABASE bank_db;
USE bank_db;

CREATE TABLE Account (
    AccountNo    INT PRIMARY KEY,
    CustomerName VARCHAR(50),
    Balance      DECIMAL(12,2),
    AccountType  VARCHAR(20)
);

INSERT INTO Account VALUES
(1001, 'Amit',  20000, 'Savings'),
(1002, 'Rahul', 15000, 'Current'),
(1003, 'Neha',      0, 'Savings'),
(1004, 'Priya', 30000, 'Savings'),
(1005, 'Karan', 10000, 'Current');

-- Deposit Rs. 5000 into account 1001.
UPDATE Account SET Balance = Balance + 5000 WHERE AccountNo = 1001;

-- Withdraw Rs. 2000 from account 1002.
UPDATE Account
SET Balance = Balance - 2000
WHERE AccountNo = 1002 AND Balance >= 2000;

-- Delete account having zero balance.
DELETE FROM Account WHERE Balance = 0;

SELECT * FROM Account;


============================================================
SET 2 - Q7. COLLEGE ADMISSION SYSTEM
============================================================
CREATE DATABASE admission_db;
USE admission_db;

CREATE TABLE Admission (
    AdmissionID INT PRIMARY KEY,
    StudentName VARCHAR(50),
    Course      VARCHAR(50),
    Fees        DECIMAL(10,2),
    Status      VARCHAR(20)
);

INSERT INTO Admission VALUES
(1, 'Amit',  'CSE',  80000, 'Pending'),
(2, 'Neha',  'IT',   75000, 'Pending'),
(3, 'Rahul', 'ENTC', 70000, 'Confirmed'),
(4, 'Priya', 'CSE',  80000, 'Cancelled'),
(5, 'Karan', 'IT',   75000, 'Pending');

UPDATE Admission SET Status = 'Confirmed' WHERE AdmissionID IN (1, 2);

DELETE FROM Admission WHERE Status = 'Cancelled';

SELECT * FROM Admission WHERE Status = 'Confirmed';
```

```
============================================================
SET 2 - Q8. HOTEL RESERVATION SYSTEM
============================================================
CREATE DATABASE hotel_reservation_db;
USE hotel_reservation_db;

CREATE TABLE Booking (
    BookingID     INT PRIMARY KEY,
    GuestName     VARCHAR(50),
    RoomNo        INT,
    Amount        DECIMAL(10,2),
    BookingStatus VARCHAR(20)
);

INSERT INTO Booking VALUES
(1, 'Amit',  101, 3000, 'Active'),
(2, 'Neha',  102, 3500, 'Active'),
(3, 'Rahul', 103, 2500, 'Cancelled'),
(4, 'Priya', 104, 4000, 'Active'),
(5, 'Karan', 105, 3200, 'Active');

UPDATE Booking SET RoomNo = 110 WHERE BookingID = 1;
UPDATE Booking SET Amount = 3800 WHERE BookingID = 2;

DELETE FROM Booking WHERE BookingStatus = 'Cancelled';

SELECT * FROM Booking WHERE BookingStatus = 'Active';


============================================================
SET 2 - Q9. LIBRARY DATABASE AUTHORIZATION
============================================================
CREATE DATABASE library_auth_db;
USE library_auth_db;

CREATE TABLE Book (
    BookID INT PRIMARY KEY,
    Title  VARCHAR(100),
    Price  DECIMAL(10,2)
);

INSERT INTO Book VALUES
(1, 'DBMS', 500),
(2, 'SQL',  400);

-- Run as administrator. Replace the password with your own.
CREATE USER 'library_user'@'localhost' IDENTIFIED BY 'Lib@12345';
GRANT SELECT ON library_auth_db.Book TO 'library_user'@'localhost';

SHOW GRANTS FOR 'library_user'@'localhost';

-- Connect separately as library_user:
SELECT * FROM library_auth_db.Book;

-- This UPDATE should fail because UPDATE permission was not granted.
UPDATE library_auth_db.Book SET Price = 600 WHERE BookID = 1;

-- Reconnect as administrator:
REVOKE SELECT ON library_auth_db.Book FROM 'library_user'@'localhost';
SHOW GRANTS FOR 'library_user'@'localhost';


============================================================
SET 2 - Q10. EMPLOYEE DATABASE AUTHORIZATION
============================================================
CREATE DATABASE employee_auth_db;
USE employee_auth_db;

CREATE TABLE Employee (
    EmpID   INT PRIMARY KEY,
```

```
    EmpName VARCHAR(50),
    Salary  DECIMAL(10,2)
);

INSERT INTO Employee VALUES (1, 'Amit', 40000);

-- Run as administrator.
CREATE USER 'hr_user'@'localhost' IDENTIFIED BY 'Hr@12345';
GRANT SELECT, INSERT ON employee_auth_db.Employee TO 'hr_user'@'localhost';

SHOW GRANTS FOR 'hr_user'@'localhost';

-- Connect separately as hr_user:
INSERT INTO employee_auth_db.Employee VALUES (2, 'Neha', 45000);
SELECT * FROM employee_auth_db.Employee;

-- This DELETE should fail because DELETE permission was not granted.
DELETE FROM employee_auth_db.Employee WHERE EmpID = 2;

-- Reconnect as administrator:
REVOKE INSERT ON employee_auth_db.Employee FROM 'hr_user'@'localhost';
SHOW GRANTS FOR 'hr_user'@'localhost';


============================================================
SET 2 - Q11. HOSPITAL DATABASE AUTHORIZATION
============================================================
CREATE DATABASE hospital_auth_db;
USE hospital_auth_db;

CREATE TABLE Patient (
    PatientID   INT PRIMARY KEY,
    PatientName VARCHAR(50),
    Age         INT
);

INSERT INTO Patient VALUES (1, 'Rahul', 25);

-- Run as administrator.
CREATE USER 'reception_user'@'localhost' IDENTIFIED BY 'Rec@12345';
GRANT SELECT, UPDATE ON hospital_auth_db.Patient TO 'reception_user'@'localhost';

SHOW GRANTS FOR 'reception_user'@'localhost';

-- Connect separately as reception_user:
SELECT * FROM hospital_auth_db.Patient;
UPDATE hospital_auth_db.Patient SET Age = 26 WHERE PatientID = 1;
SELECT * FROM hospital_auth_db.Patient;

-- Reconnect as administrator:
REVOKE UPDATE ON hospital_auth_db.Patient FROM 'reception_user'@'localhost';
SHOW GRANTS FOR 'reception_user'@'localhost';

-- Connect as reception_user again; this UPDATE should fail.
UPDATE hospital_auth_db.Patient SET Age = 27 WHERE PatientID = 1;


============================================================
SET 2 - Q12. STUDENT RESULT ANALYSIS
============================================================
CREATE DATABASE result_db;
USE result_db;

CREATE TABLE Student (
    StudentID  INT PRIMARY KEY,
    Name       VARCHAR(50),
    Department VARCHAR(50),
    Marks      INT
);

INSERT INTO Student VALUES
(1, 'Amit',  'CSE',  85),
```

```
(2, 'Neha',  'IT',   78),
(3, 'Rahul', 'CSE',  65),
(4, 'Priya', 'IT',   92),
(5, 'Karan', 'ENTC', 72),
(6, 'Sneha', 'ENTC', 88);

-- Students scoring above 75, highest marks first.
SELECT * FROM Student
WHERE Marks > 75
ORDER BY Marks DESC;

-- Highest, lowest and average marks.
SELECT MAX(Marks) AS HighestMarks,
       MIN(Marks) AS LowestMarks,
       AVG(Marks) AS AverageMarks
FROM Student;

-- Average marks department-wise.
SELECT Department, AVG(Marks) AS AverageMarks
FROM Student
GROUP BY Department;


============================================================
SET 2 - Q13. EMPLOYEE SALARY ANALYSIS
============================================================
CREATE DATABASE salary_db;
USE salary_db;

CREATE TABLE Employee (
    EmpID      INT PRIMARY KEY,
    Name       VARCHAR(50),
    Department VARCHAR(50),
    Salary     DECIMAL(10,2)
);

INSERT INTO Employee VALUES
(1, 'Amit',  'IT',      60000),
(2, 'Neha',  'HR',      45000),
(3, 'Rahul', 'IT',      50000),
(4, 'Priya', 'Finance', 55000),
(5, 'Karan', 'HR',      35000),
(6, 'Sneha', 'Finance', 70000);

-- Employees earning more than Rs. 40,000.
SELECT * FROM Employee WHERE Salary > 40000;

-- Total salary expenditure department-wise.
SELECT Department, SUM(Salary) AS TotalSalary
FROM Employee
GROUP BY Department;

-- Employees earning more than the overall average salary.
SELECT * FROM Employee
WHERE Salary > (SELECT AVG(Salary) FROM Employee);


============================================================
SET 2 - Q14. CUSTOMER ORDER MANAGEMENT
============================================================
CREATE DATABASE order_db;
USE order_db;

CREATE TABLE Customer (
    CustomerID   INT PRIMARY KEY,
    CustomerName VARCHAR(50)
);

CREATE TABLE Orders (
    OrderID    INT PRIMARY KEY,
    CustomerID INT,
    Amount     DECIMAL(10,2),
```

```
    FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID)
);

INSERT INTO Customer VALUES
(1, 'Amit'),
(2, 'Neha'),
(3, 'Rahul'),
(4, 'Priya'),
(5, 'Karan');

INSERT INTO Orders VALUES
(101, 1, 1500),
(102, 2, 2500),
(103, 1, 1000),
(104, 3, 3000),
(105, 4, 2000),
(106, 2, 1200);

-- Customer names with order amounts.
SELECT Customer.CustomerName, Orders.Amount
FROM Customer
INNER JOIN Orders ON Customer.CustomerID = Orders.CustomerID;

-- Total order amount customer-wise.
SELECT Customer.CustomerID, Customer.CustomerName,
       COALESCE(SUM(Orders.Amount), 0) AS TotalAmount
FROM Customer
LEFT JOIN Orders ON Customer.CustomerID = Orders.CustomerID
GROUP BY Customer.CustomerID, Customer.CustomerName;

-- Customers who have not placed any orders.
SELECT Customer.CustomerName
FROM Customer
LEFT JOIN Orders ON Customer.CustomerID = Orders.CustomerID
WHERE Orders.OrderID IS NULL;


============================================================
SET 2 - Q15. LIBRARY BOOK ANALYSIS
============================================================
CREATE DATABASE book_analysis_db;
USE book_analysis_db;

CREATE TABLE Book (
    BookID   INT PRIMARY KEY,
    Title    VARCHAR(100),
    Category VARCHAR(50),
    Price    DECIMAL(10,2)
);

INSERT INTO Book VALUES
(1, 'DBMS',          'Education',   500),
(2, 'SQL Basics',    'Education',   400),
(3, 'Python',        'Programming', 700),
(4, 'C Programming', 'Programming', 600),
(5, 'Java',          'Programming', 800),
(6, 'Novel',         'Fiction',     350);

-- Books priced between Rs. 300 and Rs. 800.
SELECT * FROM Book WHERE Price BETWEEN 300 AND 800;

-- Number of books in each category.
SELECT Category, COUNT(*) AS NumberOfBooks
FROM Book
GROUP BY Category;

-- Categories having more than two books.
SELECT Category, COUNT(*) AS NumberOfBooks
FROM Book
GROUP BY Category
HAVING COUNT(*) > 2;
```

```
============================================================
SET 2 - Q16. DEPARTMENT AND EMPLOYEE ANALYSIS
============================================================
CREATE DATABASE dept_analysis_db;
USE dept_analysis_db;

CREATE TABLE Department (
    DeptID   INT PRIMARY KEY,
    DeptName VARCHAR(50)
);

CREATE TABLE Employee (
    EmpID   INT PRIMARY KEY,
    EmpName VARCHAR(50),
    Salary  DECIMAL(10,2),
    DeptID  INT,
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
);

INSERT INTO Department VALUES
(1, 'IT'),
(2, 'HR'),
(3, 'Finance');

INSERT INTO Employee VALUES
(101, 'Amit',  60000, 1),
(102, 'Neha',  55000, 1),
(103, 'Rahul', 40000, 2),
(104, 'Priya', 45000, 2),
(105, 'Karan', 70000, 3),
(106, 'Sneha', 65000, 3);

-- Employee names with department names.
SELECT Employee.EmpName, Department.DeptName
FROM Employee
JOIN Department ON Employee.DeptID = Department.DeptID;

-- Average salary for each department.
SELECT Department.DeptName, AVG(Employee.Salary) AS AverageSalary
FROM Department
JOIN Employee ON Department.DeptID = Employee.DeptID
GROUP BY Department.DeptID, Department.DeptName;

-- Departments having average salary greater than Rs. 50,000.
SELECT Department.DeptName, AVG(Employee.Salary) AS AverageSalary
FROM Department
JOIN Employee ON Department.DeptID = Employee.DeptID
GROUP BY Department.DeptID, Department.DeptName
HAVING AVG(Employee.Salary) > 50000;


============================================================
SET 2 - Q17. BANKING ACCOUNT VIEW AND AUDIT TRIGGER
============================================================
CREATE DATABASE bank_audit_db;
USE bank_audit_db;

CREATE TABLE Account (
    AccountNo    INT PRIMARY KEY,
    CustomerName VARCHAR(50),
    Balance      DECIMAL(12,2)
);

INSERT INTO Account VALUES
(1001, 'Amit',  60000),
(1002, 'Neha',  30000),
(1003, 'Rahul', 80000);

CREATE VIEW high_balance_accounts AS
SELECT * FROM Account WHERE Balance > 50000;
```

```
CREATE TABLE AccountAudit (
    AuditID    INT AUTO_INCREMENT PRIMARY KEY,
    AccountNo  INT,
    OldBalance DECIMAL(12,2),
    NewBalance DECIMAL(12,2),
    ChangeTime TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

DELIMITER //

CREATE TRIGGER account_balance_audit
AFTER UPDATE ON Account
FOR EACH ROW
BEGIN
    IF OLD.Balance <> NEW.Balance THEN
        INSERT INTO AccountAudit (AccountNo, OldBalance, NewBalance)
        VALUES (OLD.AccountNo, OLD.Balance, NEW.Balance);
    END IF;
END //

DELIMITER ;

UPDATE Account SET Balance = 70000 WHERE AccountNo = 1001;

SELECT * FROM high_balance_accounts;
SELECT * FROM AccountAudit;


============================================================
SET 2 - Q18. EMPLOYEE SALARY VIEW AND TRIGGER
============================================================
CREATE DATABASE employee_audit_db;
USE employee_audit_db;

CREATE TABLE Employee (
    EmpID      INT PRIMARY KEY,
    EmpName    VARCHAR(50),
    Salary     DECIMAL(10,2),
    Department VARCHAR(50)
);

INSERT INTO Employee VALUES
(101, 'Amit',  50000, 'IT'),
(102, 'Neha',  45000, 'HR'),
(103, 'Rahul', 60000, 'IT');

CREATE VIEW it_employees AS
SELECT * FROM Employee WHERE Department = 'IT';

CREATE TABLE SalaryAudit (
    AuditID    INT AUTO_INCREMENT PRIMARY KEY,
    EmpID      INT,
    OldSalary  DECIMAL(10,2),
    NewSalary  DECIMAL(10,2),
    ChangeTime TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

DELIMITER //

CREATE TRIGGER salary_update_audit
AFTER UPDATE ON Employee
FOR EACH ROW
BEGIN
    IF OLD.Salary <> NEW.Salary THEN
        INSERT INTO SalaryAudit (EmpID, OldSalary, NewSalary)
        VALUES (OLD.EmpID, OLD.Salary, NEW.Salary);
    END IF;
END //

DELIMITER ;

UPDATE Employee SET Salary = 55000 WHERE EmpID = 101;
```

```
SELECT * FROM it_employees;
SELECT * FROM SalaryAudit;


============================================================
SET 2 - Q19. INVENTORY STOCK MONITORING
============================================================
CREATE DATABASE inventory_db;
USE inventory_db;

CREATE TABLE Product (
    ProductID   INT PRIMARY KEY,
    ProductName VARCHAR(50),
    Price       DECIMAL(10,2),
    Stock       INT
);

INSERT INTO Product VALUES
(101, 'Laptop',   50000, 15),
(102, 'Mouse',      500,  5),
(103, 'Keyboard',  1000,  8);

CREATE VIEW low_stock_products AS
SELECT * FROM Product WHERE Stock < 10;

CREATE TABLE StockAudit (
    AuditID    INT AUTO_INCREMENT PRIMARY KEY,
    ProductID  INT,
    OldStock   INT,
    NewStock   INT,
    ChangeTime TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

DELIMITER //

CREATE TRIGGER stock_update_audit
AFTER UPDATE ON Product
FOR EACH ROW
BEGIN
    IF OLD.Stock <> NEW.Stock THEN
        INSERT INTO StockAudit (ProductID, OldStock, NewStock)
        VALUES (OLD.ProductID, OLD.Stock, NEW.Stock);
    END IF;
END //

DELIMITER ;

UPDATE Product SET Stock = 7 WHERE ProductID = 101;

SELECT * FROM low_stock_products;
SELECT * FROM StockAudit;


============================================================
SET 2 - Q20. BANK MONEY TRANSFER
============================================================
CREATE DATABASE bank_transaction_db;
USE bank_transaction_db;

CREATE TABLE Account (
    AccountNo    INT PRIMARY KEY,
    CustomerName VARCHAR(50),
    Balance      DECIMAL(12,2)
) ENGINE=InnoDB;

INSERT INTO Account VALUES
(1001, 'Amit', 20000),
(1002, 'Neha', 15000);

-- Transfer Rs. 5,000 from account 1001 to account 1002.
START TRANSACTION;
```

```
UPDATE Account
SET Balance = Balance - 5000
WHERE AccountNo = 1001 AND Balance >= 5000;

UPDATE Account
SET Balance = Balance + 5000
WHERE AccountNo = 1002;

SELECT * FROM Account;

COMMIT;

SELECT * FROM Account;

-- Withdraw Rs. 2,000 and cancel it using ROLLBACK.
START TRANSACTION;

UPDATE Account
SET Balance = Balance - 2000
WHERE AccountNo = 1001 AND Balance >= 2000;

SELECT * FROM Account;

ROLLBACK;

SELECT * FROM Account;


============================================================
SET 2 - Q21. INVENTORY TRANSACTION MANAGEMENT
============================================================
CREATE DATABASE inventory_transaction_db;
USE inventory_transaction_db;

CREATE TABLE Product (
    ProductID   INT PRIMARY KEY,
    ProductName VARCHAR(50),
    Stock       INT
) ENGINE=InnoDB;

INSERT INTO Product VALUES
(101, 'Laptop',   20),
(102, 'Mouse',    30),
(103, 'Keyboard', 25);

START TRANSACTION;

UPDATE Product SET Stock = Stock - 5 WHERE ProductID = 101;

SAVEPOINT sp1;

UPDATE Product SET Stock = Stock - 3 WHERE ProductID = 102;

SELECT * FROM Product;

ROLLBACK TO SAVEPOINT sp1;

COMMIT;

SELECT * FROM Product;


============================================================
SET 2 - Q22. EMPLOYEE BONUS TRANSACTION
============================================================
CREATE DATABASE bonus_transaction_db;
USE bonus_transaction_db;

CREATE TABLE Employee (
    EmpID   INT PRIMARY KEY,
    EmpName VARCHAR(50),
```

```
    Salary  DECIMAL(10,2)
) ENGINE=InnoDB;

INSERT INTO Employee VALUES
(101, 'Amit',  40000),
(102, 'Neha',  45000),
(103, 'Rahul', 50000);

START TRANSACTION;

UPDATE Employee SET Salary = Salary + 2000 WHERE EmpID = 101;

SAVEPOINT sp1;

UPDATE Employee SET Salary = Salary + 3000 WHERE EmpID = 102;

SELECT * FROM Employee;

ROLLBACK TO SAVEPOINT sp1;

COMMIT;

SELECT * FROM Employee;


============================================================
SET 2 - Q23. BANK INTEREST CALCULATION
============================================================
CREATE DATABASE bank_routines_db;
USE bank_routines_db;

CREATE TABLE Account (
    AccountNo    INT PRIMARY KEY,
    CustomerName VARCHAR(50),
    Balance      DECIMAL(12,2)
);

INSERT INTO Account VALUES
(1001, 'Amit',  50000),
(1002, 'Neha',  75000),
(1003, 'Rahul', 30000);

DELIMITER //

CREATE FUNCTION calculate_interest(p_balance DECIMAL(12,2))
RETURNS DECIMAL(12,2)
DETERMINISTIC
NO SQL
BEGIN
    RETURN p_balance * 0.04;
END //

CREATE PROCEDURE get_account_details(IN p_account_no INT)
BEGIN
    SELECT * FROM Account WHERE AccountNo = p_account_no;
END //

DELIMITER ;

SELECT calculate_interest(50000) AS Interest;
CALL get_account_details(1001);


============================================================
SET 2 - Q24. EMPLOYEE BONUS CALCULATION
============================================================
CREATE DATABASE employee_routines_db;
USE employee_routines_db;

CREATE TABLE Employee (
    EmpID   INT PRIMARY KEY,
    EmpName VARCHAR(50),
```

```
    Salary  DECIMAL(10,2)
);

INSERT INTO Employee VALUES
(101, 'Amit',  40000),
(102, 'Neha',  45000),
(103, 'Rahul', 50000);

DELIMITER //

CREATE FUNCTION calculate_bonus(p_salary DECIMAL(10,2))
RETURNS DECIMAL(10,2)
DETERMINISTIC
NO SQL
BEGIN
    RETURN p_salary * 0.10;
END //

CREATE PROCEDURE increase_salary(
    IN p_emp_id INT,
    IN p_amount DECIMAL(10,2)
)
BEGIN
    UPDATE Employee
    SET Salary = Salary + p_amount
    WHERE EmpID = p_emp_id;
END //

DELIMITER ;

SELECT calculate_bonus(40000) AS Bonus;

CALL increase_salary(101, 2000);

SELECT * FROM Employee;


============================================================
SET 2 - Q25. ONLINE SHOPPING DISCOUNT CALCULATION
============================================================
CREATE DATABASE shopping_routines_db;
USE shopping_routines_db;

CREATE TABLE Product (
    ProductID   INT PRIMARY KEY,
    ProductName VARCHAR(50),
    Price       DECIMAL(10,2)
);

INSERT INTO Product VALUES
(1, 'Laptop',   50000),
(2, 'Mouse',      500),
(3, 'Keyboard',  1000);

DELIMITER //

CREATE FUNCTION calculate_discount(p_price DECIMAL(10,2))
RETURNS DECIMAL(10,2)
DETERMINISTIC
NO SQL
BEGIN
    RETURN p_price * 0.15;
END //

CREATE PROCEDURE update_price(
    IN p_product_id INT,
    IN p_new_price DECIMAL(10,2)
)
BEGIN
    UPDATE Product
    SET Price = p_new_price
    WHERE ProductID = p_product_id;
```

```
END //

DELIMITER ;

SELECT calculate_discount(1000) AS Discount;

CALL update_price(1, 48000);

SELECT * FROM Product;


============================================================
END OF COMBINED SQL QUESTION BANK
============================================================
```